

软件设计说明书

前端学习 APP

版本: V1.0

日期: 2026-05-30

状态: 正式

1. 引言

1.1 目的

本文档旨在描述”前端学习 APP”(CodeQuest)的软件设计, 为开发人员提供系统架构、模块划分、接口设计、数据结构等方面的设计指导, 作为系统实现的技术依据。

1.2 适用范围

本文档适用于开发人员和审查人员。

1.3 术语

术语	定义
MVP	Minimum Viable Product, 最小可行产品
Contract-first	先定义 API 契约再实现的开发模式
JWT	JSON Web Token, 用于身份认证
XP	经验值, 学习者完成学习行为获得的积分

2. 总体设计

2.1 系统架构

系统采用前后端分离的三端架构:

前端学习APP系统		
学习者端(Flutter)	管理后台(Vue 3)	
REST API (Rust + Salvo)		
PostgreSQL 数据库		

2.2 技术栈

后端: Rust 1.75+, Salvo 0.89, SQLx 0.8, PostgreSQL 15+, jsonwebtoken, bcrypt

移动端: Flutter 3.x, Dart 3.2+, GetX

管理后台: Vue 3.3+, Element Plus 2.4+, Pinia, TypeScript 5.0+, Vite 5.x

API 契约: OpenAPI 3.0.3 (单一真源 5800+ 行)

2.3 设计原则

1. **Contract-first**: 所有 API 变更必须先更新 OpenAPI 契约
2. **统一响应信封**: 成功 {data, meta}, 错误 {error, meta}
3. **禁止 unwrap**: Handler 层必须使用 map_err + StatusError
4. **按领域分文件**: Handler 按业务领域拆分

2.4 模块划分

子系统	模块	职责
后端	auth	注册、登录、JWT、刷新、登出
后端	course	课程列表/详情
后端	chapter	章节 CRUD
后端	exercise	练习 CRUD
后端	submission	代码提交与同步判题
后端	challenge	挑战 CRUD 与提交
后端	daily_challenge	每日挑战与自动复制
后端	user	个人资料与统计
后端	social	好友、动态、搜索
后端	leaderboard	排行榜 (全局/好友/课程)
后端	reward	XP 流水与徽章
后端	progress	学习进度追踪
后端	ai_help	AI 辅助请求
后端	admin	25+ 管理端点
移动端	19 个视图页面	课程、挑战、社交、个人中心等
管理后台	9 个管理页面	课程、用户、挑战、反馈等

3. 接口设计

3.1 通信协议

- 协议: HTTPS (生产) / HTTP (开发)
- 数据格式: JSON
- 认证方式: JWT Bearer Token (access 15min + refresh 7d)

3.2 API 概览

认证接口: register, learner/login, admin/login, refresh, logout

学习者接口: courses, chapters, exercises, submissions, challenges, daily-challenges,

profile、stats、friends、leaderboards、rewards、progress、ai/help

管理接口: stats、courses、challenges、exercises、users、feedback、moderation、announcements、configs、daily-challenges

3.3 统一响应格式

成功响应: { "data": T, "meta": { "request_id", "server_time" } }

列表响应: { "data": { "items": [...], "meta": { "page", "page_size", "total", "has_more" } }, "meta": { ... } }

错误响应: { "error": { "code", "message", "field_errors", "details", "retryable" }, "meta": { ... } }

4. 数据结构设计

4.1 数据库 ER 概要

系统包含 21 张业务表, 划分为以下领域:

领域	表名	说明
用户认证	accounts、account_roles、sessions	账号、角色、会话
用户资料	learner_profiles、admin_profiles	学习者和管理员资料
课程内容	courses、chapters、exercises、exercise_options、exercise_test_cases	课程-章节-练习层次
挑战系统	challenges、challenge_stages、challenge_attempts	挑战关卡与记录
每日挑战	daily_challenges、daily_challenge_records	每日一题
学习记录	submissions、course_progress、xp_ledger	提交、进度、XP
社交	friend_relations、social_activities	好友、动态
奖励	badges、learner_badges	徽章定义与获得
排行榜	leaderboard_snapshots	排行榜快照
管理	feedback_tickets、moderation_cases、announcements、system_configs、audit_logs	反馈、审核、公告、配置、审计

4.2 核心表设计

accounts: id(UUID)、email、password_hash、default_role(learner/admin)、account_status(active/suspend)

courses: id、course_code、title、difficulty、status(draft/published/archived)、esti-

mated_minutes

chapters: id, course_id, title, learning_content_markdown, order_index, unlock_rule

exercises: id, chapter_id, exercise_type(single_choice/coding), starter_code, pass_score

submissions: id, exercise_id, learner_id, source_code, judge_status(pending/judging/success/error), score

4.3 枚举类型

系统使用 PostgreSQL 自定义枚举，包括：account_status、role_type、course_status、difficulty_level、exercise_type、judge_status、friend_status 等 20+ 个枚举。

5. 模块设计

5.1 认证模块

基于 JWT 双令牌机制：access token（15 分钟）用于 API 鉴权，refresh token（7 天）用于令牌续期。密码使用 bcrypt 哈希存储。Session 表管理多设备登录，支持同一账号最多 3 个设备同时在线。

5.2 判题模块

采用同步判题模式：提交代码 → 创建 pending 记录 → 调用 JudgeService → 更新结果 → 返回。支持 mock 模式（开发调试）和 DeepSeek AI 模式（生产环境）。判题通过后自动调用 XpService 发放 XP。

5.3 每日挑战模块

每日 0 点刷新挑战题目。如果当天无生效挑战，系统自动从最近生效挑战复制一个。学习者在限时（默认 5 分钟）内完成作答，记录结果并更新连续学习天数。

5.4 AI 辅助模块

支持四种请求类型：ErrorExplanation（错误解释）、Hint（提示）、ErrorLocation（错误定位）、CorrectionHint（修正提示）。集成 DeepSeek API，使用 request 异步 HTTP 调用。每日 AI 调用限额由 learner_profiles.ai_daily_limit 控制。

5.5 进度追踪模块

CourseProgress 表记录每门课程的学习进度（completed_chapter_count、progress_percent），支持章节完成标记后自动更新百分比。所有 XP 变更记录在 XpLedger 表中，确保可追溯。

5.6 排行榜模块

使用 LeaderboardSnapshot 快照表定期计算排名，支持全局、好友、课程三个维度的排行榜。快照刷新频率可根据数据量动态调整。

6. 安全设计

6.1 身份认证

- JWT 双令牌机制, access token 短时效 (15 分钟)
- refresh token 支持轮换, 防止重放攻击
- bcrypt 密码哈希, 盐值自动生成

6.2 权限控制

- 路由级别鉴权 (JWT 中间件)
- 管理员角色校验中间件
- 学习者和管理员 API 路径隔离

6.3 数据安全

- 密码哈希存储, 不存储明文
 - AuditLog 记录所有敏感管理操作
 - 会话可主动撤销 (登出)
-

7. 运行设计

7.1 初始化流程

```
tracing_subscriber → AppConfig::from_env() → db::create_pool()
→ db::run_migrations() → db::seed_dev_accounts()
→ routes::create_router() → Server::serve()
```

7.2 配置优先级

local.toml > 环境变量 (APP__ 前缀) > default.toml

7.3 日志

基于 tracing 框架, 结构化日志输出。生产环境建议收集到集中式日志平台。

8. 维护设计

8.1 数据库迁移

使用 SQLx 迁移机制, 迁移文件按数字前缀顺序执行 (001~008)。已执行的迁移文件不可修改。

8.2 契约维护

所有 API 变更必须按顺序执行: OpenAPI → mock/examples → backend impl → frontend adapters → contract tests → end-to-end verify。

文档结束
